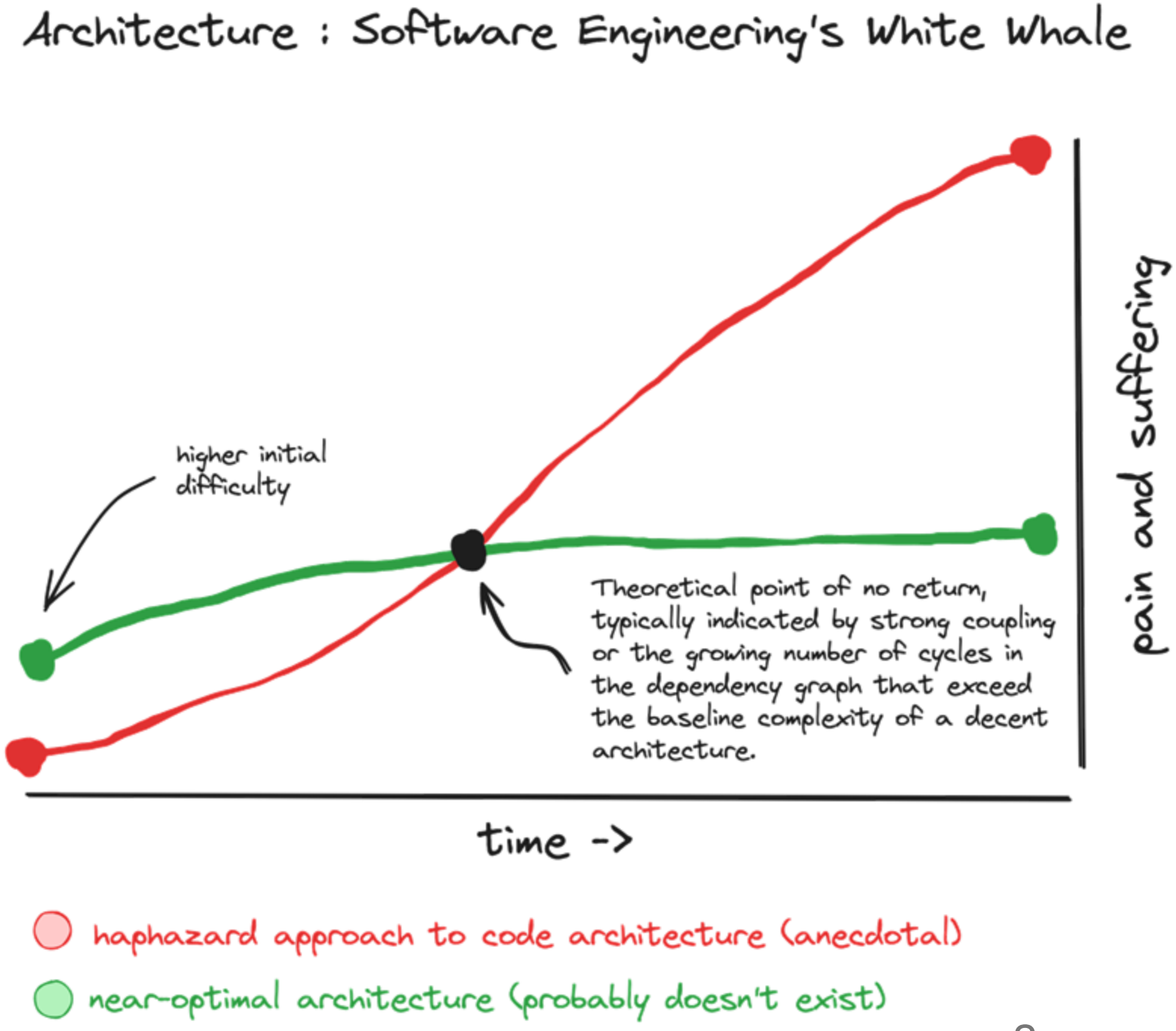
Intermezzo
Game architecture

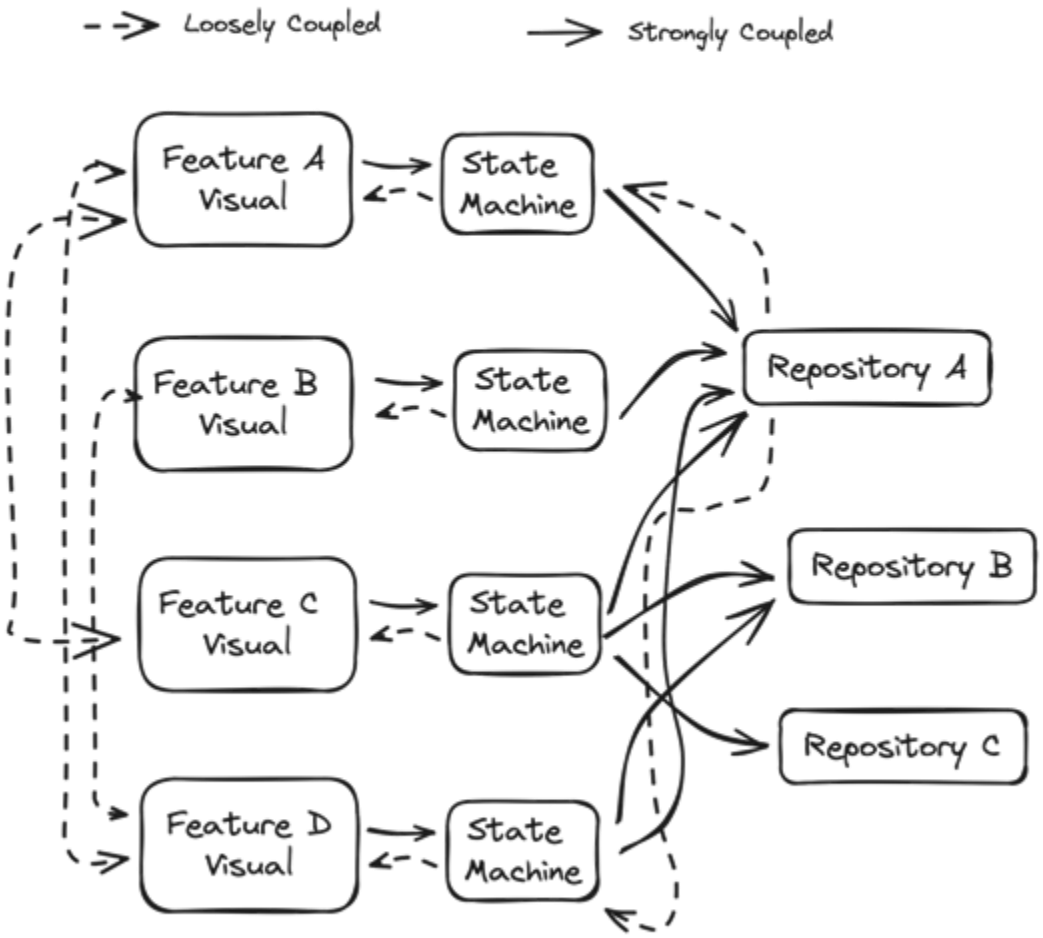
Johan Bontes

Why architecture?

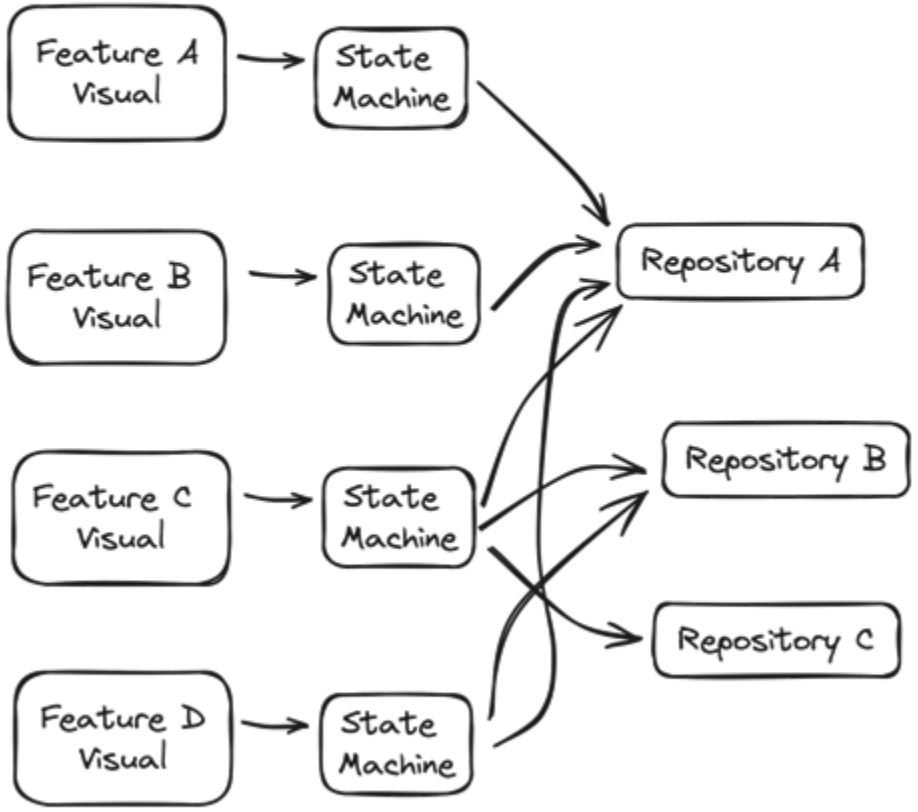


Architecture Sanity Test

1. Draw out your architecture. Use solid lines for strong coupling, and dashed lines for loose coupling.



2. Draw it again, without the loose coupling.



3. If all that remains is a directed, acyclic graph, it's at least a decent architecture.

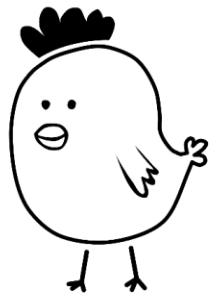
4. Refer others to your first diagram. That's your architecture!

Game Architecture

Layers of Abstraction

3 main layers:
Visuals -> Game Logic -> Data

(Although it's easier if you think of Game Logic as two different layers.)



Chickensoft — Open source tools for Godot and C#.

https://chickensoft.games

Most Abstract

1

Game Visuals Layer
(Unity MonoBehaviors, Godot Nodes, etc.)

2

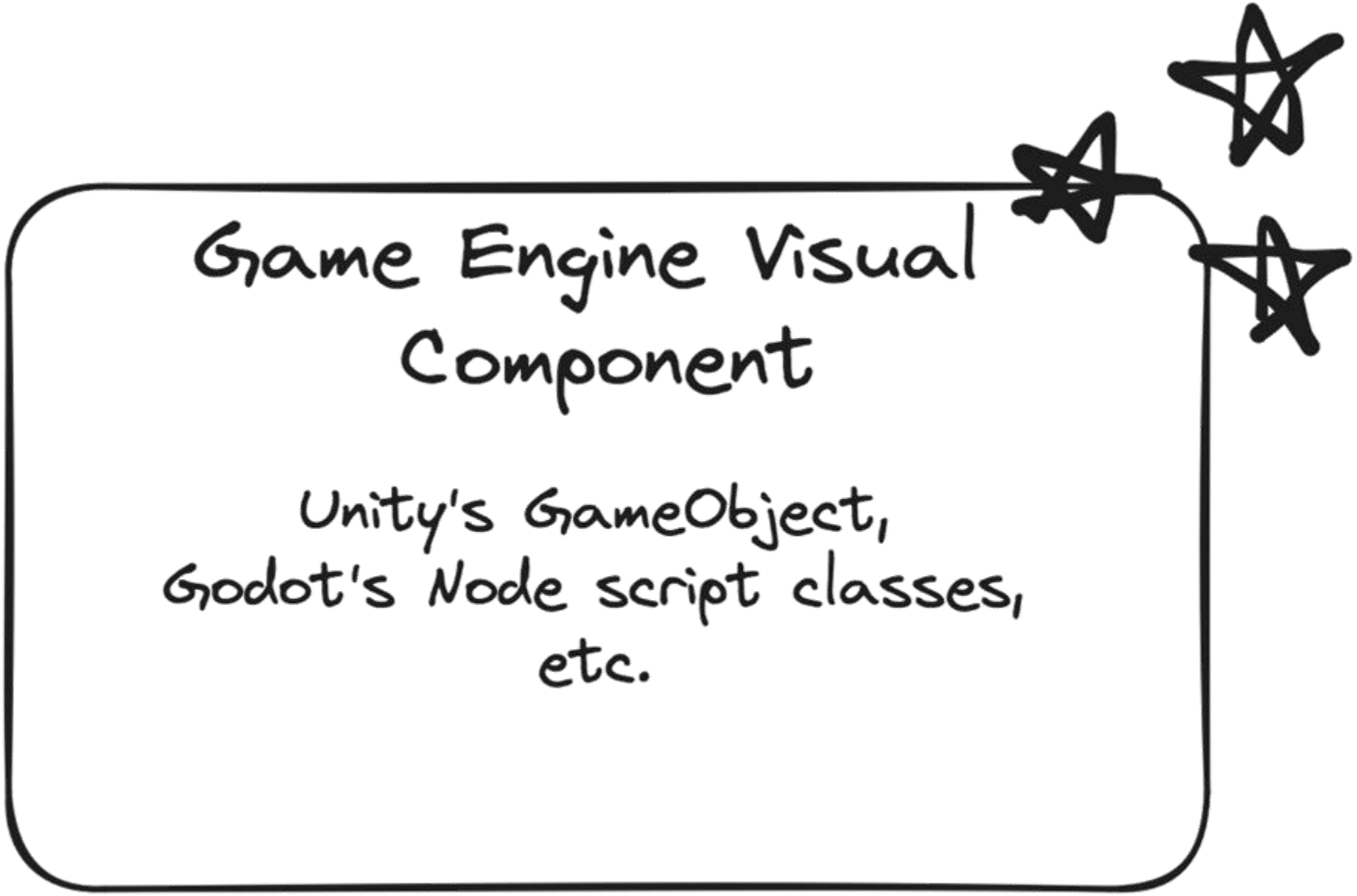
Visual Game Logic Layer
(State machines, behavior trees, etc.)

"Pure" Game Logic (Domain)
(Game logic that's not specific to a single visualization.)

Least Abstract

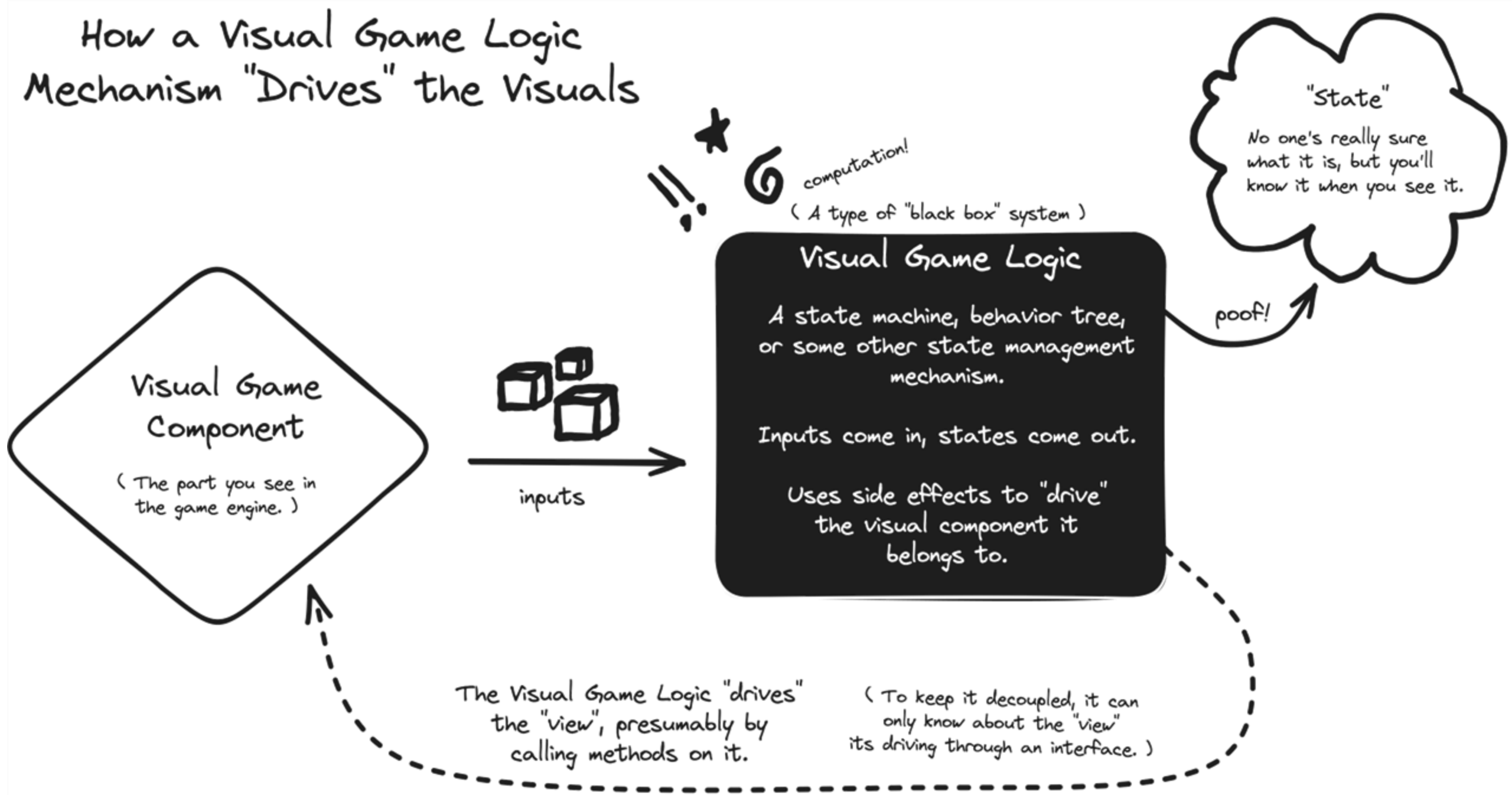
3

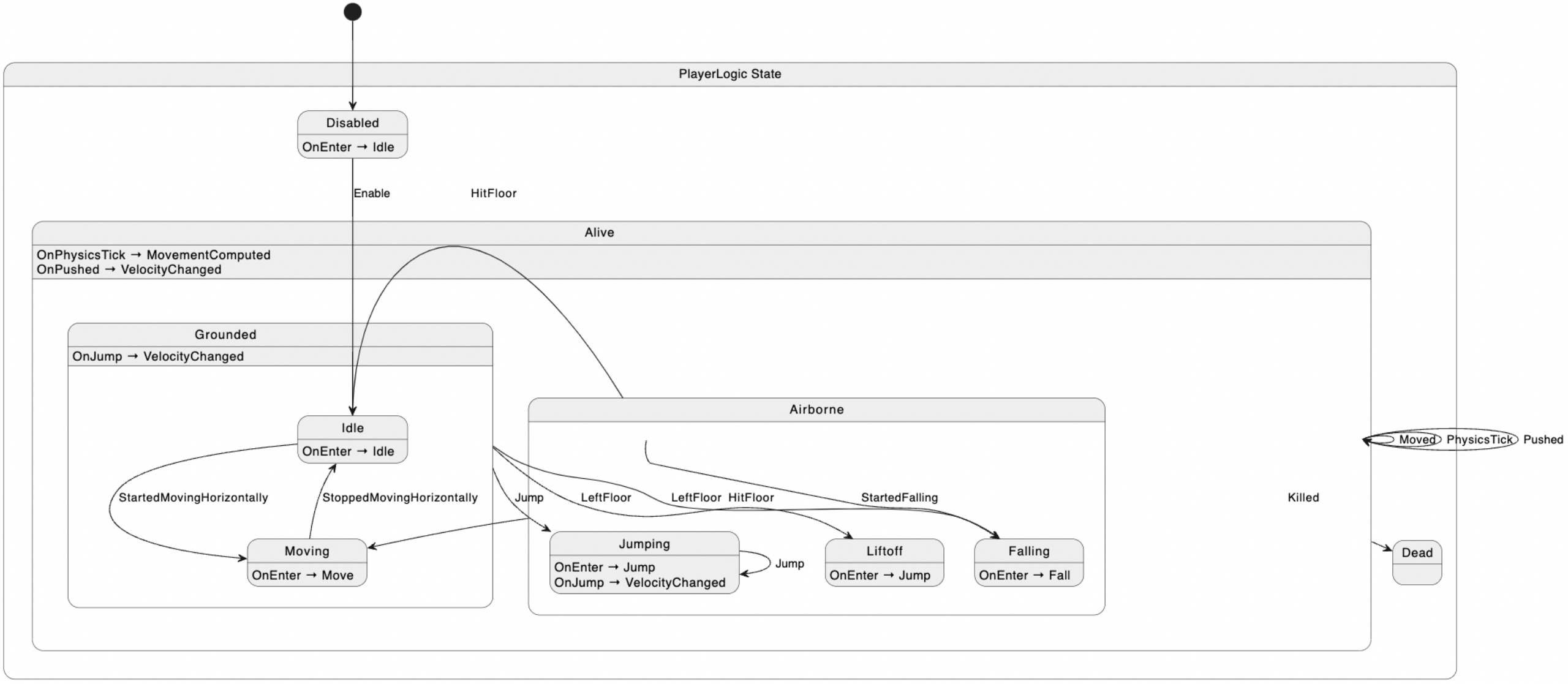
Data Layer
(Networking, persistent storage, etc)



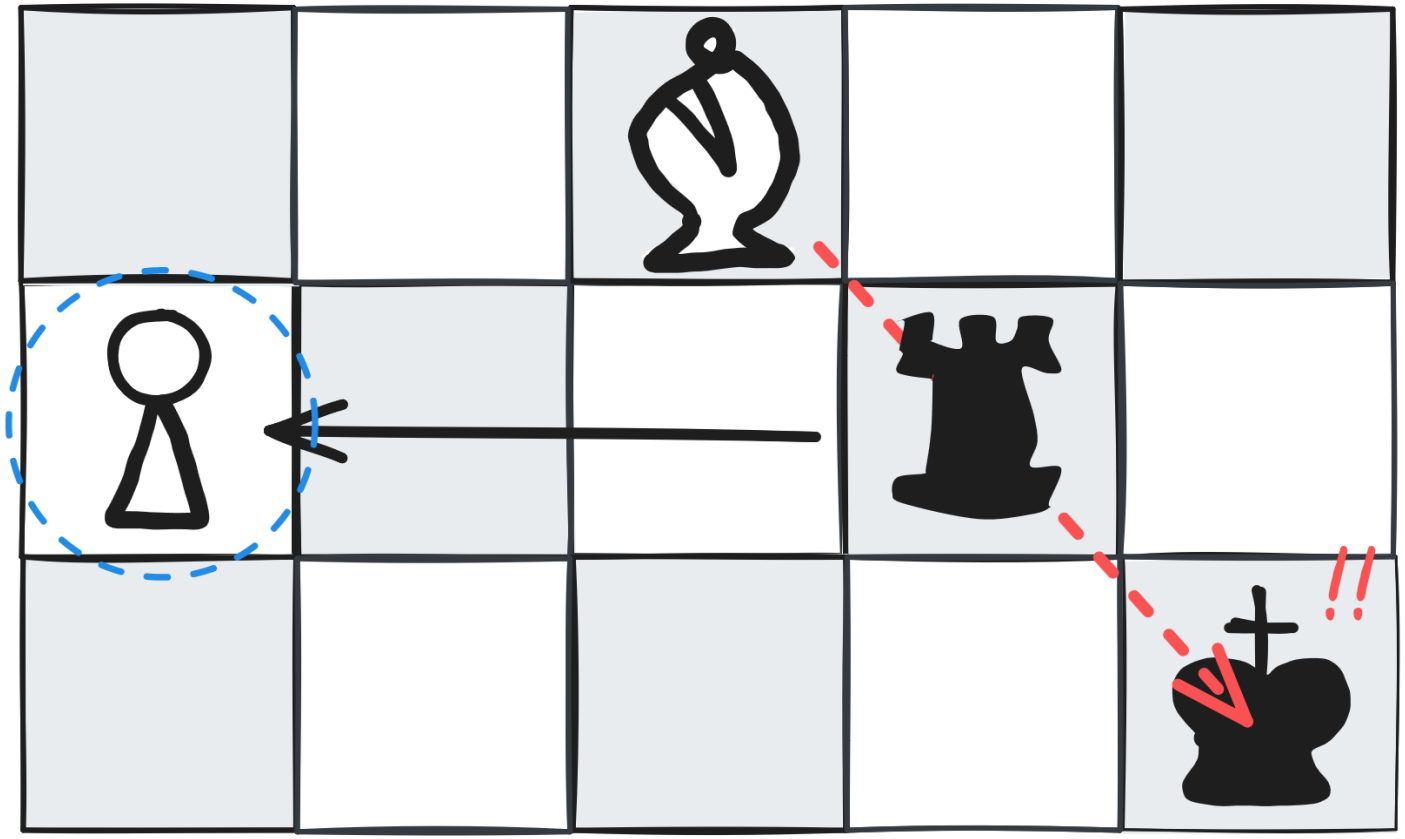
(The shiny things you actually see in-game!)

How a Visual Game Logic Mechanism "Drives" the Visuals




PlayerLogic State
Disabled
OnEnter → Idle
Enable
HitFloor
Alive
OnPhysicsTick → MovementComputed
OnPushed → VelocityChanged
Grounded
OnJump → VelocityChanged
Idle
OnEnter → Idle
StartedMovingHorizontally
StoppedMovingHorizontally
Moving
OnEnter → Move
Airborne
Jump
LeftFloor
LeftFloor HitFloor
StartedFalling
Killed
Jumping
OnEnter → Jump
OnJump → VelocityChanged
Jump
Liftoff
OnEnter → Jump
Falling
OnEnter → Fall
Moved
PhysicsTick
Pushed
Dead

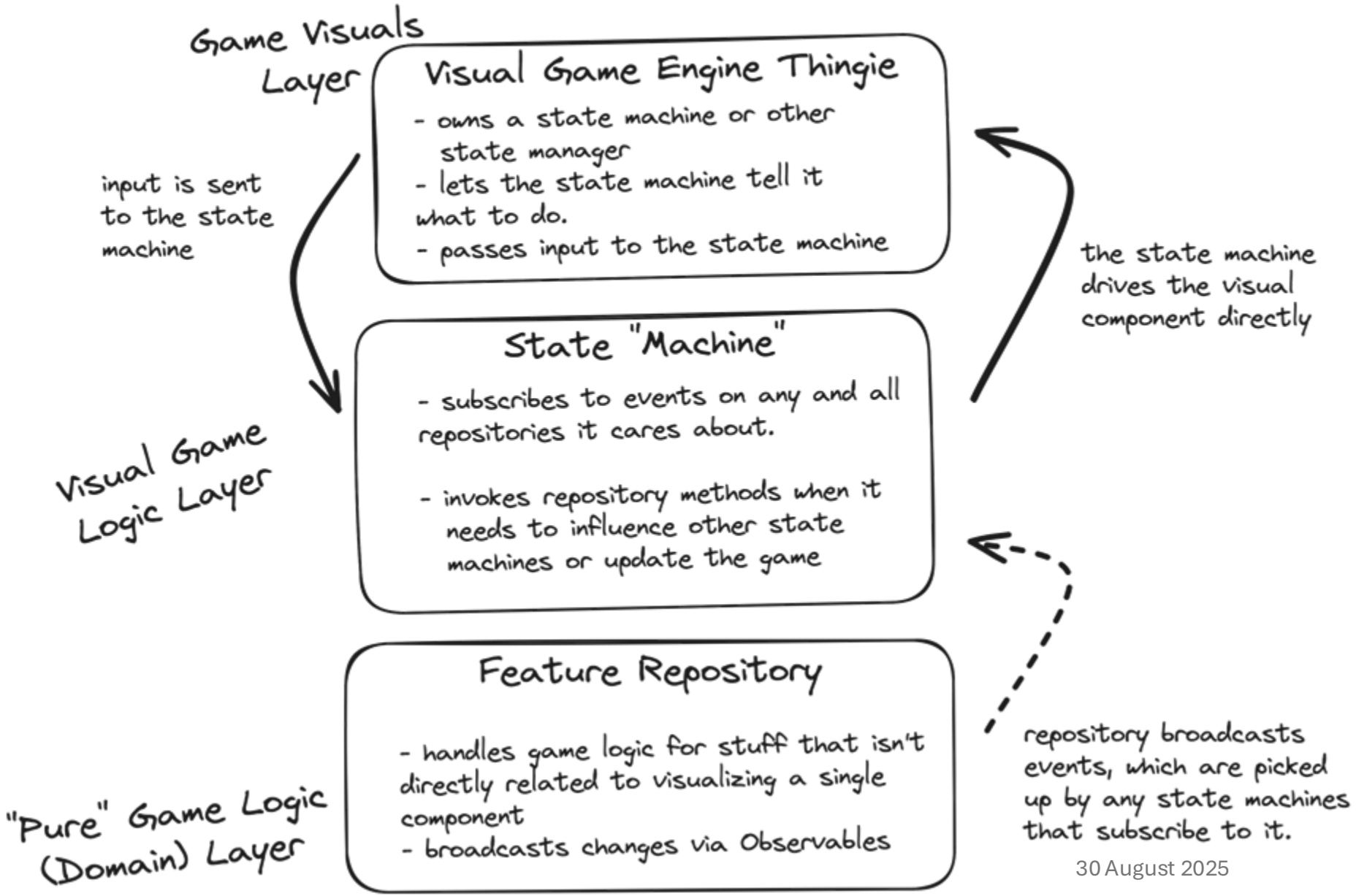
Capturing in Chess



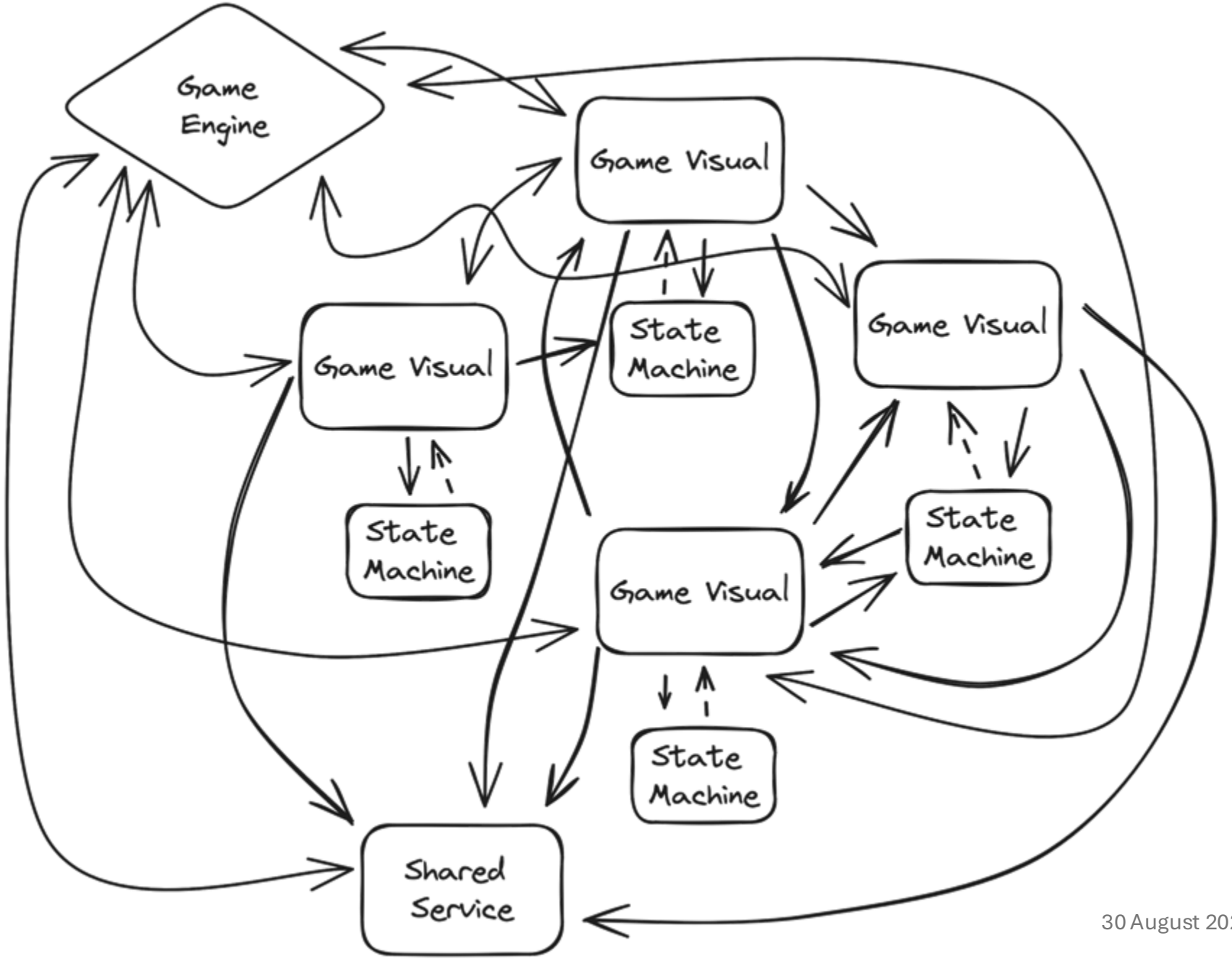
- Target is reachable by the piece.
- Moving the piece does not put the king in check.
- Capturing the target displaces the target, removing it from the board.
- The capturing piece takes the target's former position.

Separating Game Logic from Visualization

Or, how information flows uphill without strong coupling.



Not-so-layered Architecture

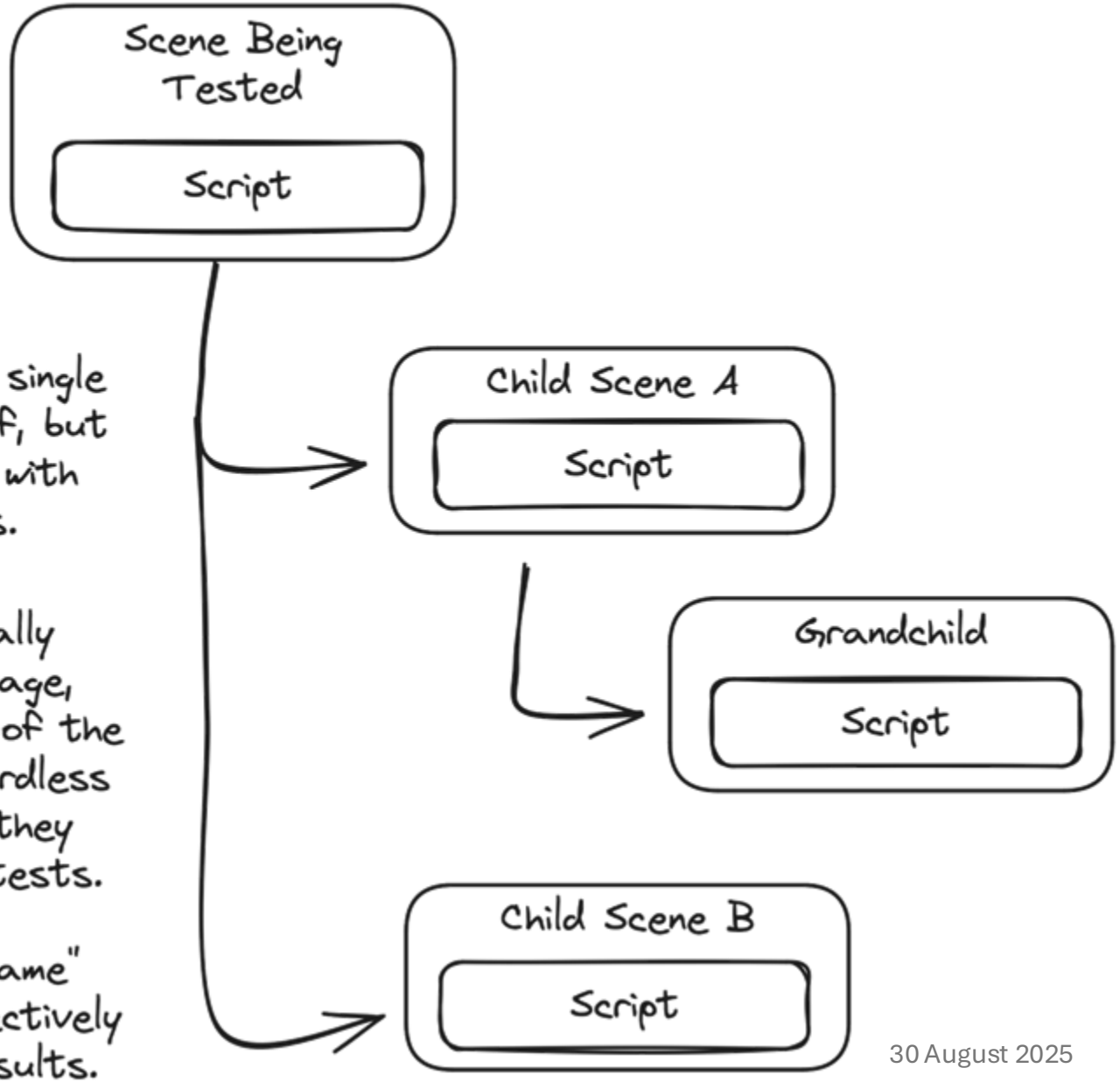


The Scene Explosion Problem

You want to test a single Godot scene by itself, but it has child scenes with their own scripts.

Testing it artificially inflates code coverage, since it covers parts of the other scripts — regardless of whether or not they have their own unit tests.

This is a way to "game" code coverage — effectively contaminating the results.



Why That One Teammate Won't Shut Up About Unit Tests

Showing Your Work

$$\frac{f(y) = 3y + 1}{\texttt{assert(f(1) == 4)}}$$

This is less about verification and more about demonstrating what you are trying to accomplish.



Documentation

Tests are a type of living documentation that won't go out of date (since your code won't compile if they did).

And, if you test everything, you can reference how to do anything.



Enforced Decoupling

Unit tests are hard to write if your code isn't decoupled, so the very act of trying to write a test will bring you to a halt (or at least slow you down) if you've made structural mistakes.



Oh, and unit tests help you verify your code does what it says it does (not what you think it does — you need integration tests or end-to-end tests for that).

Layers of Abstraction in Testing

(The further you make it into this chart, the more polarizing you will be to other engineers.)

No Tests	"we don't have time" or "it's not possible / too hard."
Unit Tests	Verifies the code does what it says it does.
Integration Tests	Verifies the code does what you think it does.
End-to-End Tests	Verifies the code does what your product manager thinks it does.
User Testing	Verifies the code does what the user thinks it does.